ARCHITECTURE & ENGINEERING REFERENCE

IgnitionChat

How the Chat Bridge, the Angular frontend, and the MCP Gateway fit together — component breakdown, authentication design, request/data flow diagrams, the test evidence gathered for each layer, and the architectural decisions behind them.

Prepared 2026-08-13 · Project root: `C:\Users\katlego.gagoopane\Documents\2026\Claude\chat-bridge`

> **Intended audience:** systems engineers and architects evaluating this design — either to operate it, extend it, or use it as a reference pattern for other backend-for-frontend (BFF) integrations in front of Claude. Sections call out the alternatives considered and why each decision was made, not just what was built.

Contents

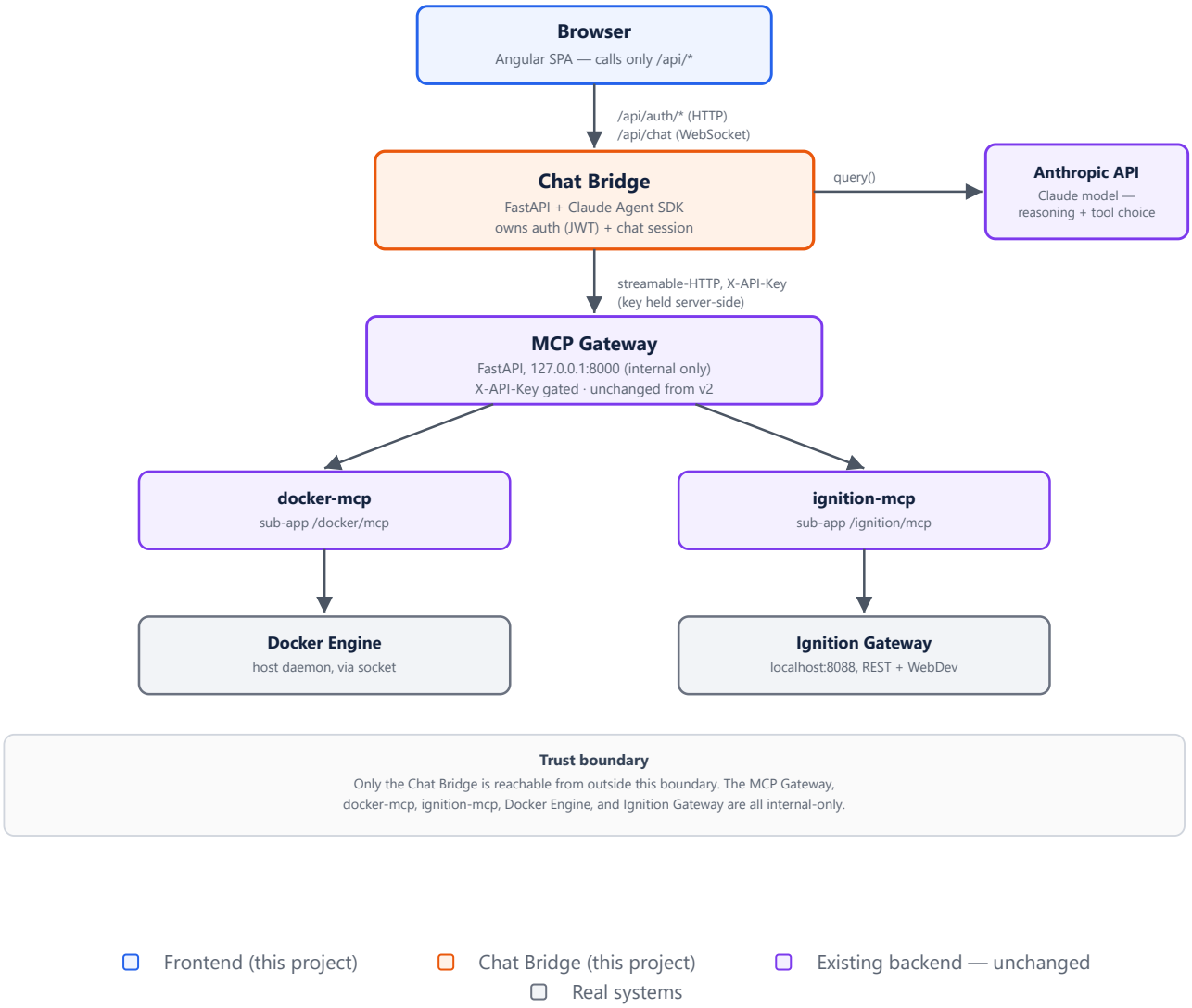
1. Executive Summary

IgnitionChat lets an operator ask natural-language questions about a live Ignition SCADA system (and, when enabled, Docker containers) from a browser, with Claude deciding which tools to call and an MCP Gateway executing them against the real systems. The project is split into three independently deployable pieces:

Project	Path	Role
frontend	`C:\...\Claude\frontend`	Angular 22 signals/zoneless SPA — sign-in/sign-up, chat UI
chat-bridge	`C:\...\Claude\chat-bridge`	Python/FastAPI backend-for-frontend — auth, session, Claude Agent SDK
backend	`C:\...\Claude\backend`	MCP Gateway + `docker-mcp` / `ignition-mcp` tool servers (pre-existing, unchanged)

The central design choice is a **backend-for-frontend (BFF)**: the browser talks to exactly one service (Chat Bridge), which alone holds the MCP Gateway's API key and brokers every call to Claude. Section 3 lays out why this shape was chosen over the alternatives that were on the table.

2. System Architecture Overview



3. Architectural Decisions

Presented as a decision record — each row states what was chosen, what else was on the table, and why the alternative lost.

Decision	Alternatives considered	Rationale
Backend-for-frontend, separate from both Angular and the Gateway	Put auth/session logic in the frontend's own Express skeleton (`frontend/server/`); call the Gateway directly from the browser	A browser-facing Gateway would put container-creation and live-SCADA-write capability one API key away from the internet. A BFF gives one process to secure, one place to hold the Gateway's key, and keeps the Angular app a pure client with no server-side auth logic of its own.
httpOnly cookie for session, not a token the client stores	Return the JWT in the response body and have Angular attach it	A query-param token leaks into server logs, browser history, and Referer headers, and is readable by any XSS payload. An httpOnly cookie is invisible to JS, and the

	manually to the WebSocket URL as a query param	browser attaches it to the WebSocket upgrade automatically — `AuthService` never touches the token at all.
`tools=[]` on every Claude Agent SDK session	Leave the SDK's default tool set enabled and rely on `allowed_tools` alone	`allowed_tools` only auto-approves tools — it does not restrict which ones exist. Testing proved this: with the Gateway down, Claude fell back to `Bash`/`Grep` against the Bridge's own host. `tools=[]` removes every built-in tool so only the two MCP servers are ever reachable. See §7.2.
One `ClaudeSDKClient` per WebSocket connection	A single shared client multiplexing all users; a connection pool	Conversation state in the SDK is per-client. One client per connection gives each browser tab an isolated conversation for free, and failure in one session can't affect another — confirmed under concurrent same-user sessions (§10.4).
Mock in-memory user store, no database	Stand up Postgres/SQLite now	Matches the course's own reference screenshots (a single seeded user) and keeps the Bridge stateless and disposable. Explicitly flagged as a placeholder, not a final decision — see §14.
Chat Bridge can serve the built Angular app itself (optional)	Always require a separate static host/CDN in front	Keeps "the Bridge is the only public entry point" true by construction in the simplest deployment, without mandating a second service. Purely additive — unset `FRONTEND_DIST_PATH` and the Bridge is API-only, as it is in local dev.

4. Chat Bridge — Component Breakdown

4.1 File map

`src/chat_bridge/app.py`
FastAPI application object and every route: `GET /health`, `POST /api/auth/signup`, `POST /api/auth/login`, `WebSocket /api/chat`. Also conditionally mounts the built Angular app as static files with SPA-route fallback (§12), registered last so it never shadows an `/api/*` route.

`src/chat_bridge/auth.py`
The mock user store (in-memory `dict[email, argon2 hash]`, seeded with `test@angular-university.io`), password hashing/verification via `argon2-cffi`, and JWT issuance/verification via `PyJWT` (HS256, `exp` validated on decode).

`src/chat_bridge/claude_service.py`
Everything that talks to Claude and the Gateway: builds `ClaudeAgentOptions` (MCP server URLs + `X-API-Key`, `tools=[]`, `allowed_tools`), the `ChatSession` context manager wrapping one `ClaudeSDKClient` per connection, and `serialize_message` / `_serialize_block`, which turn SDK message objects into the JSON frames sent to the browser.

`src/chat_bridge/config.py`

A single `pydantic-settings` `Settings` object, loaded once from `.env` / the process environment: Gateway URL + key, bind host/port, JWT secret/expiry, cookie security flag, optional Claude model override, optional `frontend_dist` path.

`src/chat_bridge/__init__.py`

`main()` — the `chat-bridge` console script entry point; runs `uvicorn` against `chat_bridge.app:app` on `settings.host` / `settings.port`.

`pyproject.toml, uv.lock, .python-version`

`uv`-managed project metadata and locked dependencies: `fastapi`, `uvicorn[standard]`, `claude-agent-sdk`, `pydantic-settings`, `pyjwt`, `argon2-cffi`. Pinned to Python 3.12, matching the rest of the backend.

`.env.example, .gitignore, .dockerignore`

Template for the real `.env` (never committed) documenting every setting in `config.py`; ignore rules keep the venv, caches, and secrets out of both git and Docker build contexts.

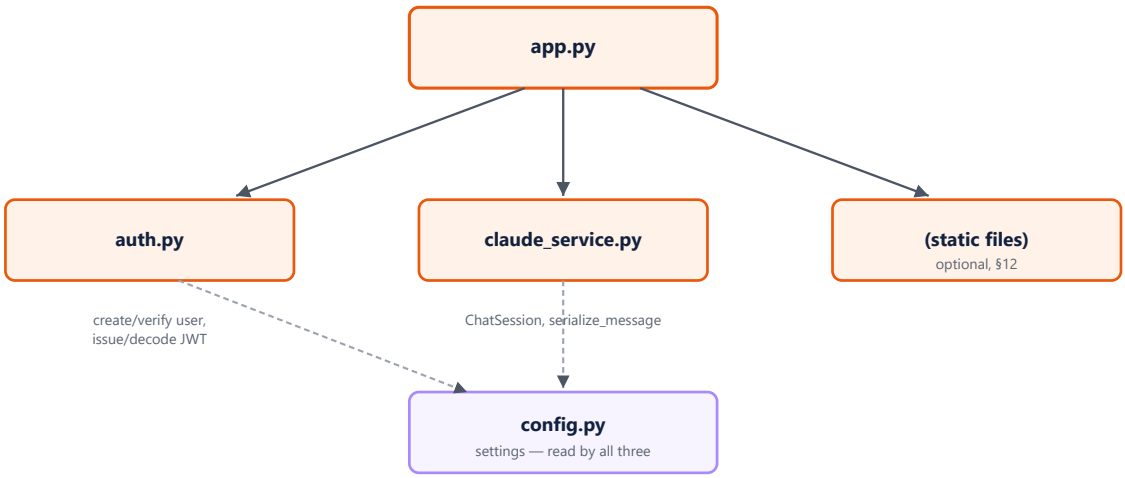
`Dockerfile, docker-compose.yml (in backend/mcp-gateway/)`

Container image for the Bridge, and the compose file that runs it alongside the Gateway with only the Bridge's port published. See §12.

`ARCHITECTURE.md`

The living design record this document is generated from — ownership, resolved/open questions, and the security note in §7.2.

4.2 Internal module diagram



Solid arrows: `app.py` imports and calls into each module for its routes. Dashed arrows: every module reads the shared `settings` object.

4.3 Request lifecycle inside the Bridge

1. **HTTP request or WebSocket upgrade arrives** at `app.py`'s route table.
2. **Auth routes** (`/api/auth/login`, `/api/auth/signup`) call into `auth.py`, get back a JWT, and set it as the `access_token` cookie on the response.
3. **The `/api/chat` WebSocket route** reads that same cookie, calls `auth.py`'s `decode_access_token`, and rejects the handshake (HTTP 403, before `accept()`) if it's missing, malformed, or expired.
4. On success it opens a `ChatSession` from `claude_service.py` — one `ClaudeSDKClient`, configured via `config.py`'s settings, for the lifetime of that connection.
5. Each inbound JSON frame becomes a call to `session.send(prompt)`; each outbound `SDKMessage` is serialized and pushed back over the socket as its own JSON frame.

5. Authentication Architecture

5.1 Components involved

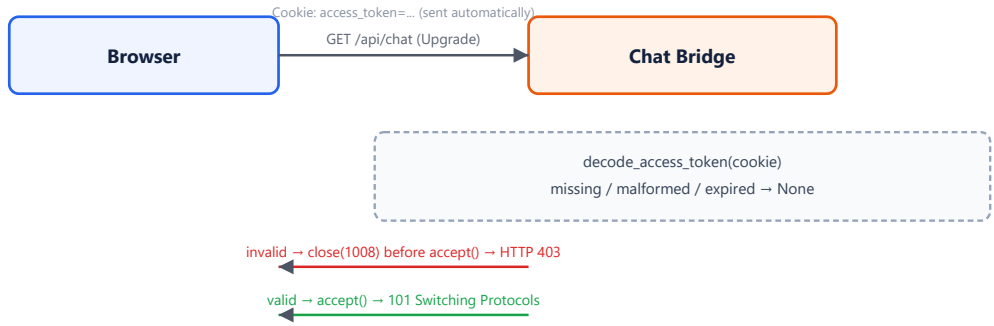
Component	Layer	Responsibility
`Auth` (Angular service)	Frontend	Calls `/api/auth/login/signup` via `fetch`; tracks a local `isAuthenticated` signal. Never reads or stores the token itself.
`authGuard` (Angular `CanActivateFn`)	Frontend	Blocks `/chat` unless `Auth.isAuthenticated()` is true; redirects to `/sign-in` otherwise.
`SignIn` / `SignUp` components	Frontend	Signal Forms UI that calls `Auth.login()/signup()` and navigates to `/chat` on success.
`/api/auth/login`, `/api/auth/signup`	Chat Bridge	Verify/create the mock user, issue a JWT, set it as an httpOnly cookie.
`auth.py`	Chat Bridge	Password hashing (argon2), JWT encode/decode (PyJWT, HS256).
`/api/chat` WebSocket gate	Chat Bridge	Reads the cookie on every connection attempt; rejects before `accept()` if invalid.

5.2 Sign-in sequence

1. User submits the sign-in form. `SignIn` calls `Auth.login({email, password})`.
2. `Auth` issues `fetch('/api/auth/login', {credentials: 'include', ...})`.
3. Chat Bridge verifies the password against the argon2 hash in `auth.py`'s in-memory store. On failure: `401`, body `{"detail": "Invalid email or password"}`.
4. On success: Bridge issues a JWT (`sub`=email, `iat`, `exp`) and returns it two ways — in the JSON body (unused by the client) and as a `Set-Cookie: access_token=...; HttpOnly; SameSite=Lax` header.

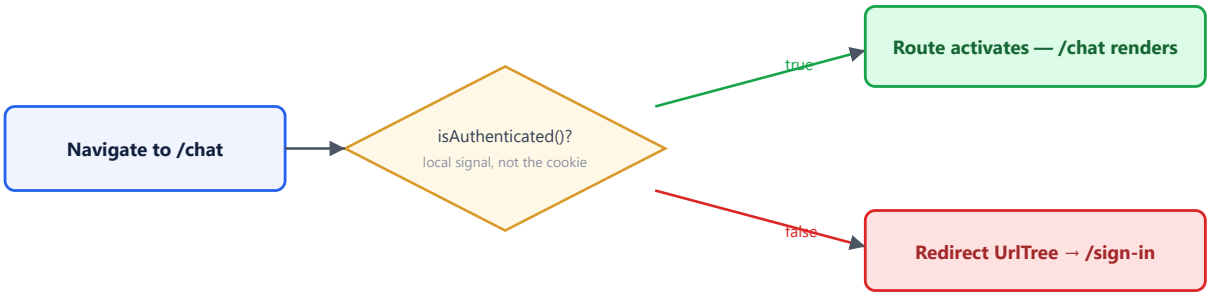
5. The browser stores the cookie itself; `Auth` just flips its local `isAuthenticated` signal to `true` on a `2xx` response.
6. `SignIn` calls `router.navigateByUrl('/chat')`. `authGuard` checks `isAuthenticated()` (already true) and allows the SPA navigation.

5.3 WebSocket auth handshake



Verified directly: expired/garbage cookies reject at the handshake (403) before a `ChatSession` — and therefore before any Claude/Gateway resource — is ever created.

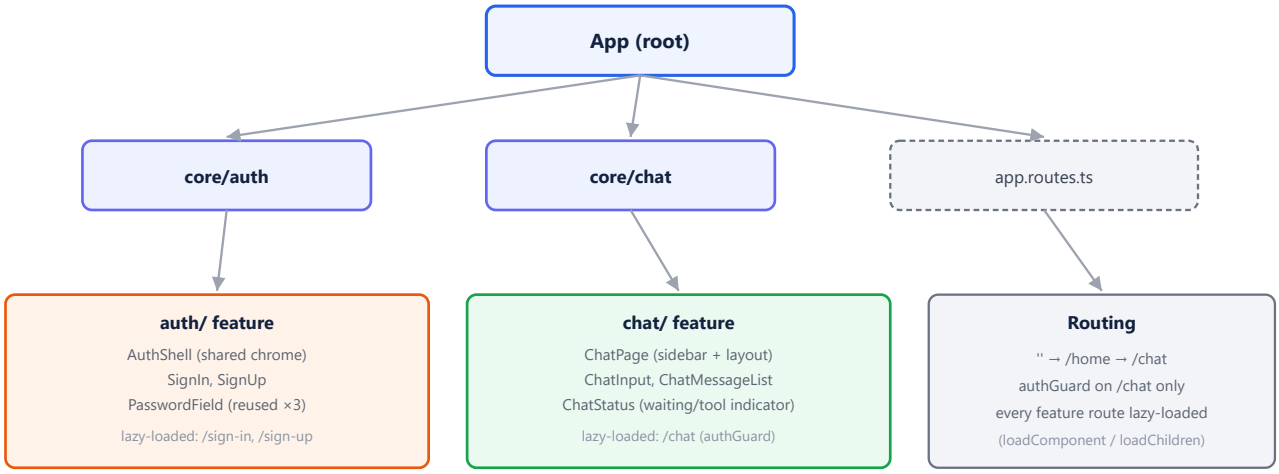
5.4 Route guard flow



Known limitation, by design trade-off: the guard checks a local Angular signal, not the httpOnly cookie (which JS cannot read). A page reload resets that signal to `false` even if the cookie is still valid server-side, so the user is asked to sign in again after any refresh. Closing this gap cleanly needs a session-check endpoint (e.g. `GET /api/auth/me`) — tracked as an open item in §14, not fixed here to avoid scope creep into backend changes that weren't requested.

6. Angular Frontend Structure

6.1 Module / component tree



auth/ and chat/ inject core/auth's Auth service; chat/ additionally injects core/chat's Chat service.
Neither feature imports the other directly — they only share state through the two core/ services.

6.2 core/ services

Service	Type	Responsibility
`Auth` (`core/auth/auth.ts`)	`@Service()`, root-provided	`login()`/`signup()`/`logout()` over `fetch`; exposes `isAuthenticated` as a readonly signal.
`authGuard` (`core/auth/auth-guard.ts`)	Functional `CanActivateFn`	Gate for the `/chat` route — see §5.4.
`Chat` (`core/chat/chat.ts`)	`@Service()`, root-provided	Wraps the native `WebSocket` in a Promise/signal API — `connect()`, `send()`, `disconnect()`, `reset()` — and exposes `messages`, `connected`, `awaitingReply`, `connectionError` as signals.

Both services follow the project's own coding rules: no RxJS (native `fetch` / `WebSocket` wrapped as Promises), no manual DI in constructors (`inject()`), and state exposed as signals rather than Observables so components can bind to them directly with `computed()`.

6.3 auth/ feature

`AuthShell` is the shared branding-pane-plus-card layout, content-projected by both `SignIn` and `SignUp` so the visual chrome exists in exactly one place. `PasswordField` is a custom Signal Forms control (implements `FormValueControl<string>`) providing the show/hide toggle, reused for the sign-in password field and both sign-up password fields. Both screens are built with **Signal Forms** (`form()`, `required()`, `email()`, `validate()` for the sign-up password-confirmation cross-field check), per `src/CLAUDE.md`'s project rule — not Reactive or template-driven forms.

6.4 chat/ feature

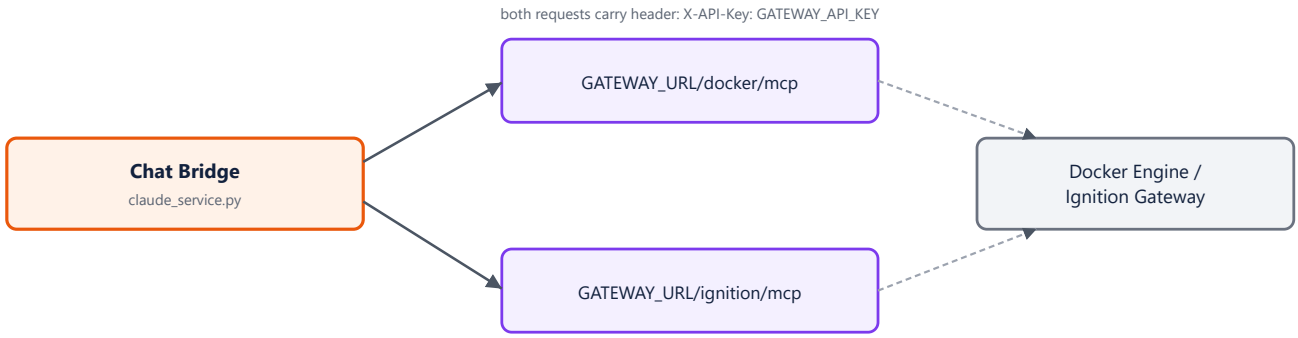
`ChatPage` is the route target: it injects both `Auth` and `Chat`, opens the WebSocket connection on construction, and renders either the empty-state (centered input, matching the reference screenshot) or the active-conversation layout, depending on whether `Chat.messages()` is non-empty. `ChatMessageList` renders the transcript, scoping `aria-live="polite"` to only the most recent message rather than the whole list. `ChatStatus` shows the "waiting for reply" spinner with a live elapsed-seconds counter, switching to `Calling <tool-name>…` when a tool call is actually in flight — computed from the current assistant message's tool-call list. `ChatInput` is a single Signal Forms field reused for both the empty-state search bar and the active-state composer.

6.5 How the frontend reaches the Chat Bridge

Every backend call in the Angular app is required (by `src/CLAUDE.md`) to start with `/api`. In local development that convention is what makes the dev proxy transparent: `proxy.conf.json` forwards `/api/*` — including the WebSocket upgrade for `/api/chat`, which needed `"ws": true` explicitly added, since the default proxy config does not forward upgrades — to the Chat Bridge on port 8001. No other code path exists from the browser to any backend.

7. Chat Bridge → Backend (MCP Gateway) Connection

7.1 Diagram



The connection is **streamable-HTTP**, configured per MCP server in `claude_service.py`'s `_mcp_servers()`: `{"type": "http", "url": ..., "headers": {"X-API-Key": ...}}`. The key is read once from `config.py`'s `gateway_api_key` setting and never appears in any response sent to the browser — the browser only ever sees the Bridge, never the Gateway or its key.

7.2 Security hardening: restricting the tool surface

Finding (from edge-case testing, §10.5): with the MCP Gateway unreachable, Claude did not simply fail the request — it fell back to the Claude Agent SDK's **built-in Claude Code tools** (`Bash` , `Grep`), executed against the Chat Bridge's own host filesystem, and used them to go spelunking through the `chat-bridge` source tree looking for an answer. `allowed_tools=["mcp__docker__*", "mcp__ignition__*"]` only auto-approves those two tool families — it does not restrict the tool set Claude is offered. Left as originally built, any authenticated chat user had effective shell access on whatever machine runs the Bridge.

Fix, in `claude_service.py` 's `build_options()` :

```
ClaudeAgentOptions(
    mcp_servers=_mcp_servers(),
    tools=[],                # disables every built-in tool (Bash, Read, Write, Edit, Grep, ...)
    strict_mcp_config=True,  # ignore any other .mcp.json / global MCP config the CLI would load
    allowed_tools=["mcp__docker__*", "mcp__ignition__*"],
    model=settings.claude_model,
)
```

`tools` and `allowed_tools` are independent knobs in the SDK: `tools` controls which tools *exist* for the model to see at all; `allowed_tools` controls which of the tools that do exist can run *without an interactive permission prompt* — a prompt there would be no terminal on the Bridge's end to ever answer. Both are required together. Re-run after the fix, the same Gateway-down scenario correctly leaves Claude with zero tools and it replies in plain text asking for clarification, instead of reading the filesystem.

8. End-to-End Query Flow

8.1 Sequence: one chat turn, in full

1. Browser: user submits the chat form. `ChatInput` emits the prompt; `ChatPage` calls `Chat.send(prompt)` .
2. `Chat` appends an optimistic user bubble and an empty assistant placeholder to its `messages` signal, sets `awaitingReply` true, and sends `{"content": prompt}` over the already-open WebSocket.
3. Chat Bridge's `/api/chat` handler reads the frame and calls `ChatSession.send(prompt)` , which calls the SDK's `client.query(prompt)` then iterates `client.receive_response()` .
4. The Claude Agent SDK sends the conversation and tool list to **Anthropic's API**. The model decides what to say and, if needed, which MCP tool to call.
5. For a tool call, the SDK calls the **MCP Gateway** directly (streamable-HTTP, `X-API-Key` attached by the Bridge). The Gateway's middleware checks the key and routes to `docker-mcp` or `ignition-mcp` , which executes the real action.
6. Each `SDKMessage` (text, tool-use, tool-result) is serialized by `serialize_message` / `_serialize_block` and pushed to the browser as its own WebSocket frame — streaming, not one blocking response.
7. `Chat.handleFrame()` appends text to the current assistant message, adds/completes tool-call entries by matching `tool_use_id` , and updates the `messages` signal — Angular's change detection re-renders the transcript reactively.

8 A `ResultMessage` frame marks the turn complete: `awaitingReply` flips back to `false`, and `ChatStatus`'s spinner disappears.

9. This App vs. the Claude Code CLI

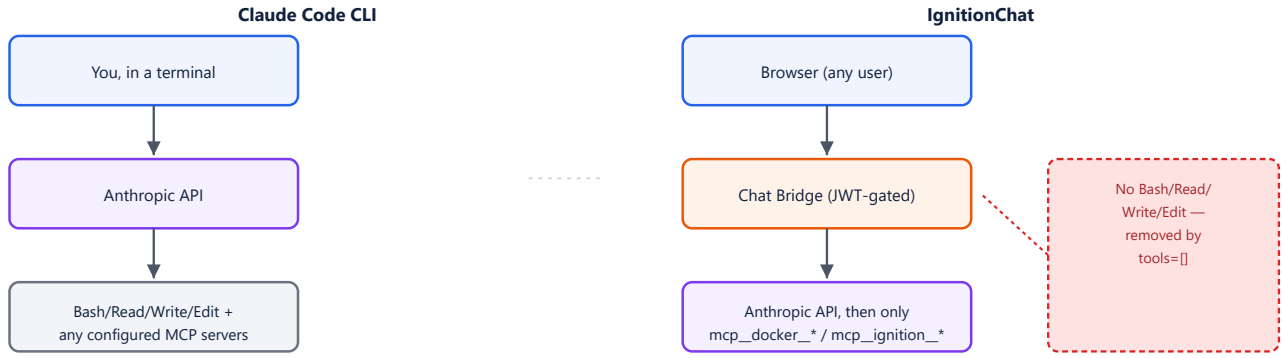
Both systems are, at their core, the **same SDK** — `claude-agent-sdk`, the library Claude Code itself is built on. Comparing them side by side is useful precisely because it shows how much of "Claude Code" is really just *SDK defaults*, and how deliberately IgnitionChat had to override those defaults for a multi-tenant server context.

Claude Code (terminal / IDE)

- One user, one trusted machine, one long-lived session
- Full built-in tool surface by default: `Bash`, `Read`, `Write`, `Edit`, `Grep`, `WebFetch`, plus any configured MCP servers
- Interactive permission prompts are answerable — a human is at the keyboard
- `cwd` is wherever the user opened their terminal — that's the point
- Session/auth is the logged-in `claude` CLI itself

IgnitionChat's Chat Bridge

- Many untrusted browser users, one shared server process
- `tools=[]` — zero built-in tools; only `mcp__docker__*` / `mcp__ignition__*` exist at all
- No terminal on the other end — `allowed_tools` must auto-approve everything reachable, since a prompt would hang forever
- `cwd` is irrelevant once `tools=[]` removes every filesystem-touching tool
- Session/auth is a per-user JWT the Bridge issues and verifies itself



10. Testing Strategy & Evidence

Five distinct kinds of testing were used, deliberately escalating from fast/isolated to slow/real:

Type	Tooling	What it covers
UNIT	Vitest, mocked `fetch`/`WebSocket`	Service and guard logic in isolation, fast feedback
VISUAL / SCRIPTED BROWSER	Headless Edge + Chrome DevTools Protocol	Real rendering against the reference screenshots; real DOM event simulation (typing, blur, click)

REAL END-TO-END	Live Ignition Gateway + MCP Gateway + authenticated `claude` CLI	The full pipeline with no mocks anywhere, including genuine Claude tool-use decisions
RESILIENCE / EDGE CASE	Same, with a component deliberately killed mid-session	Failure modes: expired auth, unreachable services, dropped connections, concurrency
SECURITY REGRESSION	Same real stack, before/after comparison	Confirms the `tools=[]` fix actually closes the gap without breaking the happy path

10.1 Unit tests (Vitest)

Spec file	Cases
`core/auth/auth.spec.ts`	login/signup success sets `isAuthenticated`; failure surfaces the server's error message and leaves it false; logout clears local state
`core/auth/auth-guard.spec.ts`	authenticated → allows; unauthenticated → returns a `UrlTree` to `/sign-in`
`core/chat/chat.spec.ts`	connect/send/streaming-frame accumulation/tool-call resolution/reset, plus the resilience additions: connect failure surfaces `connectionError` with no messages added; an unexpected close mid-turn clears `awaitingReply`; `reset()` does not falsely report a connection error
`auth/sign-in`, `auth/sign-up`, `chat/chat-page` specs	Component creation with real DI (mocked `Router`/`WebSocket` providers)

Final count: **19 tests across 6 files, all passing**, alongside a clean `ng build` production bundle.

10.2 Visual / scripted browser tests

No interactive browser tool was available in this environment, so verification used headless Microsoft Edge driven directly over the **Chrome DevTools Protocol** (`Page.navigate`, `Runtime.evaluate` to simulate real typing/blur/click via native DOM events, and `Page.captureScreenshot`). This is meaningfully different from reading the code: it exercises actual rendering, actual Signal Forms validation timing, and actual CSS.

- Sign-in / sign-up screens compared pixel-by-pixel against `ui-screenshots/*.png`
- Field-level validation: required-field errors appear only after `touched`, not on page load
- Password visibility toggle, cross-field password-confirmation mismatch on sign-up
- Chat empty-state and active-conversation layouts against their reference screenshots
- Multi-turn conversation: confirmed the composer input clears correctly on both the first *and second* message — this is how the Signal Forms reset bug in §10.5 was found

10.3 Real end-to-end test

This environment happened to have real infrastructure available: a live Ignition Gateway responding on `localhost:8088`, and an already-authenticated `claude` CLI (this session *is* Claude Code). Rather than mock these, the test ran the real MCP Gateway (`uv run gateway`, pointed at `http://localhost:8088`) and the real Chat Bridge together, then drove a real login and a real question through the actual browser UI:

> "What is the current value of the Village tag in Ignition?" → Claude called `mcp__ignition__list_tag_providers`, `browse_tags`, and `read_tags` for real, against the real Gateway, and the reply — plus all three tool-call status

badges — rendered correctly in the browser.

Docker-path tool calls were not exercised: Docker Desktop was installed but its engine was not running, and starting it was judged too invasive an action to take without being asked.

10.4 Concurrent-session test

Two independent WebSocket connections were opened for the **same user** simultaneously, each asking for a different one-word answer, to confirm the Bridge's one-`ChatSession`-per-connection design holds under concurrency with no shared mutable state:

```
await asyncio.gather(
    run_session("session A", token, "Say the word Alpha and nothing else."),
    run_session("session B", token, "Say the word Bravo and nothing else."),
)
# session B: received 4 frames
# session A: received 4 frames
```

Both sessions completed independently; neither blocked or corrupted the other.

10.5 Resilience / edge-case tests — where the real bugs were found

Scenario	Method	Result
Expired JWT on WebSocket upgrade	Signed a token with exp in the past using the Bridge's own secret; attempted the handshake	Correctly rejected, HTTP 403, before `ChatSession` creation
MCP Gateway unreachable mid-conversation	Killed the Gateway process, then asked a question that needed a tool	**FOUND** Claude fell back to `Bash`/`Grep` against the Bridge's host (§7.2) — fixed with `tools=[]`, then re-verified clean (no tool-call badges, plain-text clarification request) and the happy path re-confirmed working afterward
Chat Bridge unreachable / dropped WebSocket	Logged in for real, then killed the Bridge process mid-session (from the test harness itself, same browser tab) and attempted to send a message	**FOUND** an unhandled promise rejection and a spinner that would never stop — fixed by adding a `connectionError` signal, a visible banner in `ChatPage`, and clearing `awaitingReply` on an unexpected socket close
Signal Forms reset on repeated submit	Found incidentally while diagnosing the above: a second message in an already-active conversation	**FOUND** `ChatInput` reset its own field by writing the backing model signal directly, which `[formField]` doesn't reliably observe — masked previously because the first message always destroys/recreates the component via `@if`/`@else`. Fixed by resetting through the `FieldTree`'s own `.value.set()`; re-verified across a genuine two-turn conversation
Concurrent sessions, same user	See §10.4	No interference, no crash

Why this matters architecturally: three of five edge-case tests surfaced real defects that unit tests and code review did not — two of them (the tool fallback and the stuck-spinner/unhandled-rejection) only manifest when a

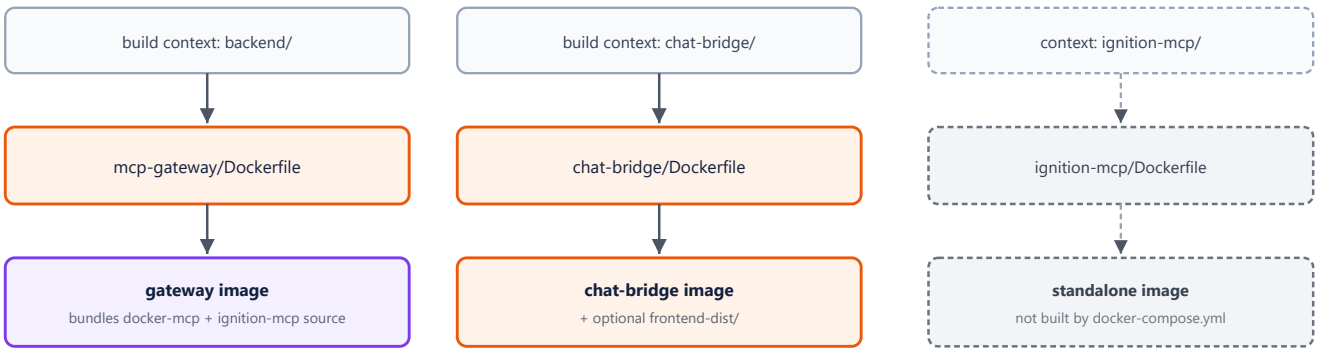
dependency actually dies mid-request, and the Signal Forms bug only manifests on a *second* interaction with an already-mounted component. This is the argument for budgeting real failure-injection testing on any BFF that brokers access to tools with real-world side effects, not just happy-path coverage.

11. Dockerfiles Across the Project

Three `Dockerfile`s exist across the backend and chat-bridge projects (the frontend has none — its build output is either handed to the Bridge or a separate static host, per §12). Only two of the three are part of this system's actual runtime topology.

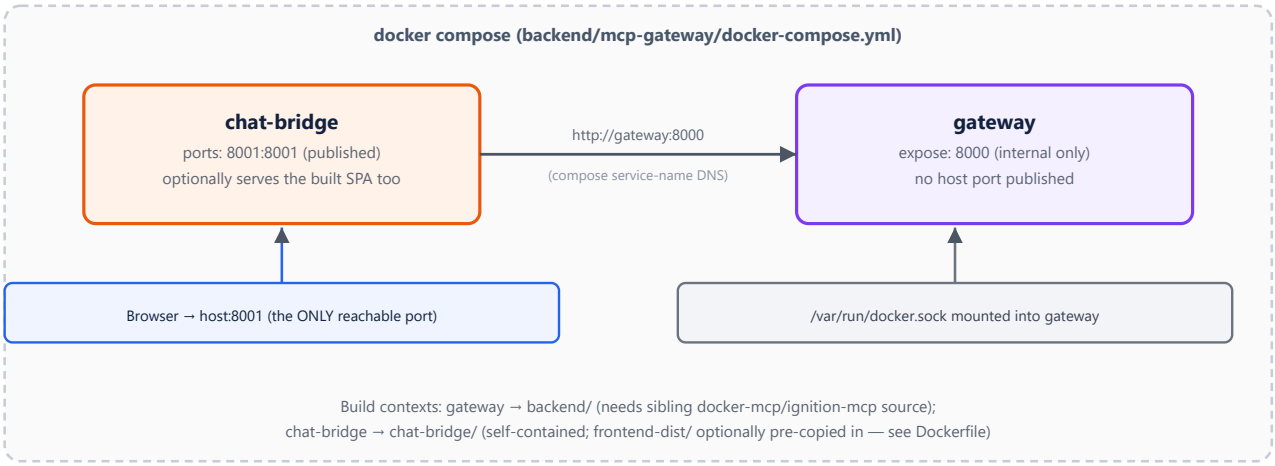
Dockerfile	Base image	Purpose	Used by this system?
`backend/mcp-gateway/Dockerfile`	`python:3.12-slim`	Builds the MCP Gateway image. Installs the Docker CLI and compose plugin (APT, from Docker's own repo) because `docker-mcp` shells out to the `docker` command rather than speaking the Engine API directly.	**YES** — built by `docker-compose.yml`
`chat-bridge/Dockerfile`	`python:3.12-slim`	Builds the Chat Bridge image. Self-contained build context; optionally bakes in a pre-built Angular static output copied into `frontend-dist/` beforehand (§4.1, §12).	**YES** — built by `docker-compose.yml`
`backend/ignition/ignition-mcp/Dockerfile`	`python:3.11-slim`	Builds `ignition-mcp` as its own **standalone** server — runs `mcp_server.py` directly, listens on its own port (8007), has its own `HEALTHCHECK`, and drops to a non-root `appuser`.	**NO** — see note below

Why the third one isn't in the runtime path: the MCP Gateway does not talk to a standalone `ignition-mcp` container over the network at all — `gateway/app.py` `import`s `ignition_mcp.server` directly as a Python library and mounts its ASGI app in-process at `/ignition` (see §7.1). `backend/ignition/ignition-mcp/Dockerfile` builds a *different*, independently runnable packaging of the same tool server — useful for running/testing `ignition-mcp` on its own, but it is not built, referenced, or started by `backend/mcp-gateway/docker-compose.yml`. Anyone extending this system should be aware two valid deployment shapes exist for `ignition-mcp` and know which one is actually live.



Solid = wired into backend/mcp-gateway/docker-compose.yml. Dashed = exists in the repo, independently runnable, not part of this system's compose graph.

12. Deployment Topology



This is the structural enforcement of the trust boundary in §2: removing `gateway`'s `ports:` mapping (replaced with `expose:`, container-network-only) makes "only the Chat Bridge is reachable from outside" true at the infrastructure level, not just by convention.

13. Startup Runbook

Local development — order matters

1. **Ignition Gateway** (if using the live-tag path) — must already be running and reachable, e.g. `http://localhost:8088`. Not managed by this project.
2. **MCP Gateway** — from `backend/mcp-gateway/`:

   ```
   IGNITION_MCP_IGNITION_GATEWAY_URL=http://localhost:8088 uv run gateway
   ```

   (The `.env`'s own default, `host.docker.internal`, only resolves from inside a container — override it when running directly on the host.) Confirm with `GET /health` on port 8000.

3. **Chat Bridge** — from `chat-bridge/`, with a real `.env` (or exported env vars) providing `GATEWAY_API_KEY` (must match the Gateway's) and `JWT_SECRET`:

   ```
   uv run uvicorn chat_bridge.app:app --host 127.0.0.1 --port 8001
   ```

   `claude-agent-sdk` itself needs either `ANTHROPIC_API_KEY` in the process environment or an already-authenticated `claude` CLI. Confirm with `GET /health` on port 8001.

4. **Angular dev server** — from `frontend/`:

   ```
   npm start
   ```

   Serves on port 4200 and proxies `/api/*` (HTTP and the `/api/chat` WebSocket) to the Chat Bridge per `proxy.conf.json`.

5. Open `http://localhost:4200`, sign in as `test@angular-university.io` / `Angular123` (or sign up), and use the chat.

Production (Docker Compose)

1. Optional: build the Angular app and copy its output into `chat-bridge/frontend-dist/` if the Bridge should serve the SPA itself (see the Dockerfile's header comment for the exact command).
2. Ensure `backend/mcp-gateway/.env` and `chat-bridge/.env` both exist, with matching `GATEWAY_API_KEY` values.
3. From `backend/mcp-gateway/`: `docker compose up --build`. Compose starts `gateway` first (internal-only) then `chat-bridge` (`depends_on: gateway`), publishing only port 8001.

14. Open Questions & Recommendations

Item	Status	Recommendation
Session doesn't survive a page reload	Known, by design trade-off (§5.4)	Add a lightweight `GET /api/auth/me` the guard can call to check the cookie server-side before deciding to redirect.
Mock in-memory user store	Placeholder	Replace with real persistence before this leaves a single-instance/dev deployment — an in-memory dict does not survive a restart or scale past one process.
Per-tool confirmation UX for high-impact actions	Not resolved	Worth revisiting given §7.2's finding — even with `tools=[]`, a write-capable Ignition or Docker tool call currently executes with no per-action human-in-the-loop step.
Conversation history beyond one connection	Not resolved	Currently one `ClaudeSDKClient` per WebSocket connection means history is lost on disconnect/reload; decide if that's acceptable for the target use case.
Docker-path tool calls	Not exercised	Re-run the real end-to-end test (§10.3) with Docker Desktop's engine running to close this gap.